

Project 3: Game

CS17 Fall '22 | Brown University

Due: 11:00 PM, December 5, 2022

Contents

1	Introduction	2
2	Assignment	3
2.1	Setup	3
2.2	Game Signature and Module	4
2.3	Player Signature and Modules	7
2.3.1	Parsing Human Input	8
2.4	The AI Player	9
2.5	How Everything Fits Together	11
3	Let's Play!	12
4	Getting Started	13
5	Testing	13
6	Notes	13
7	Handing In	14
7.1	Design Check	14
7.2	Final Handin	15
7.3	Grading	15
8	Connect 4 Tournament (Optional)	16

A Connect 4 Summary 17

B Other Optimization Method 17

“There is no finite game unless the players freely choose to play it. No one can play who is forced to play. It is an invariable principle of all play, finite and infinite, that whoever plays, plays freely. Whoever *must* play, cannot *play*.”

—*Finite and Infinite Games*, James P. Carse

“In chess too,” he said at last, “there’s a limit to the forecasts one can make. The best possible move, or the most probable one, is the one that leaves one’s opponent in the least advantageous position. That’s why one way of estimating the expediency of the next move consists simply in imagining that move has been made and then going on to analyze the game from your opponent’s point of view. That means falling back on your own resources, but this time putting yourself in your enemy’s shoes. From there, you conjecture another move and then immediately put yourself in the role of your opponent’s opponent, in other words, yourself. And so on indefinitely, as far ahead as you can. By that, I mean that I know where I’ve got to, but I don’t know how far he’s got.”

—Muñoz, *The Flanders Panel*, Arturo Pérez-Reverte

1 Introduction

For your final project, you will implement a two-player, sequential, finite-action, deterministic, zero-sum game of perfect information. Let’s define what that means:

- A **two-player** game has two players. Tic-tac-toe is a two-player game; Hearts is not.
- A **sequential** game is one in which only one player moves at a time. Monopoly is a sequential game; *Rochambeau* (i.e., rock-paper-scissors) is not.
- A **finite-action** game is one in which there is a finite number of legal moves available to a player when it is their turn to move. Battleship is a finite-action game; soccer is not.
- A **deterministic** game is one that does not depend at all on chance. Its progress is entirely a function of selected moves. Checkers is deterministic; backgammon is not.
- A **zero-sum game** is one in which what is good for one player is equally bad for the other, and vice versa. All the examples in this section are zero-sum games.
- A game of **perfect information** is one in which both players witness the entire progression of the game. Chess is a game of perfect information; poker is not.

In addition to implementing a game, you will also write a general purpose artificially intelligent (AI) player that can play any game of this sort. Your AI player will play a game using the minimax algorithm, together with a helper procedure that estimates the values of intermediate game states.

In lab, you have (or will have) seen two simple games: GoFirst and Nim. This project entails implementing a more complex game, with a more sophisticated `estimateValue` procedure and AI player.

The human player is a user interface that asks a real human which move to make. The AI player selects moves based on a generic strategy called *game-tree search*. By “generic strategy”, we mean that the approach does not depend on which game is being played. Of course, the strategy must ultimately use something specific to the game, and that specific thing is called a *static game-state evaluator*: it is a procedure that assigns a

number to each game state (called the *value* of the state) that reflects how good/bad that state is for the AI player. In our implementation, the state evaluator is called `estimateValue`. (The reason for the modifier *static* will be made clear later.)

2 Assignment

For this assignment, you will be implementing Connect 4. Note, however, that your “Game” module can be used to play *any* two-player, sequential, finite-action, deterministic, zero-sum game of perfect information. For example, this means that once you have finished implementing Connect 4, you should be able to replace your `Connect4.re` file with another game implementation, such as `Nim.re` (which you have seen, or will soon see, in lab) and all of your code should still compile and run!

Our version of Connect 4 is described at the end of this handout. Please read our description of Connect 4 carefully because our rules may differ slightly from the rules you know.

Traditionally Connect 4 is played on a board of fixed size. But, in the spirit of abstraction and elegance, we are requiring that your implementation handle boards of varying sizes (e.g., larger boards, or more pieces in play, etc.).

There are two main parts to this project.

1. You have to implement the game. This involves writing a module with data types capable of keeping track of the state of the game, and a few procedures that provide information and generate new states.
2. You have to build an AI player capable of playing the game. AI stands for artificial intelligence. Your AI player should play intelligently, where “intelligently” is understood to mean rationally.

2.1 Setup

Source code can be downloaded from the course website. It should include the following 9 files:

- `README`
- `Connect4.re`
- `HumanPlayer.re`
- `AIPlayer.re`
- `Referee.re`
- `Game.re`
- `Player.re`
- `CS17SetupGame.re`
- `ReadLineSyncTest.re` (not used for the project, only used to test the package we are about to install)

Important Note: Once you initialize a project and download the files into the `src` folder of your project, you need to download a package to allow your user to read in input. To do this, navigate in terminal to the project directory and type `npm install readline-sync`. This will install a package to allow your game to run a

human player. To test that this was installed correctly, you can compile and run `ReadLineSyncTest.bs.js`. If you can type a sentence and hit enter and get that sentence repeated back to you, then it installed correctly. Note that you won't be able to compile your project right upon downloading, because your game will not have all of the required procedures to implement the `Game` module and will throw an error. Thus, to make it compile, add failwith stubs for all required procedures.

Another note on the source code: Based on your implementation, you may require “open”-ing of different files than we have in the source code. Thus, while you should **not** alter `Game.re`, `Player.re`, or `CS17SetupGame.re`, you may need to change the headers of other files.

2.2 Game Signature and Module

For the first part of this project, you will need to understand a game abstract data type. This abstract data type captures (much of) what is common to two-person, sequential, finite-action, deterministic, zero-sum games of perfect information.

Note: The content of this section is reminiscent of (if not copied verbatim from) Lab 11.

Game Signature Here is the `Game` signature:

```
module type Game = {
  /* TYPES */
  type whichPlayer = P1 | P2;
  type status =
    | Win(whichPlayer) /* this says who wins */
    | Draw
    | Ongoing(whichPlayer); /* this says whose turn it is */
  type state;
  type move;

  /* INITIAL GAME STATE */
  let initialState: string => state;

  /* TYPE CONVERSIONS */
  let stringOfPlayer: whichPlayer => string;
  let stringOfState: state => string;
  let stringOfMove: move => string;
  let moveOfString: (string, state) => move;

  /* GAME LOGIC */
  let legalMoves: state => list(move);
  let gameStatus: state => status;
  let nextState: (state, move) => state;
  let estimateValue: state => float;
};
```

This signature has five main components:

- Types:
 - There is a variant type `whichPlayer` that specifies one of the two players.

- There is a type `status` that summarizes the status of the game (e.g. whose turn it is if the game is not over).
- One type that you must define for your particular game is `state`, which is used for representing everything about the progress of the game. How is the current board represented, and whose turn is it? For example, for Checkers, this would involve specifying for each square of the board whether the square was occupied by red or black or neither.
- Another type that you must define for your game is `move`, which is used for representing all the possible moves a player might ever be able to make. For example, a move in Nim would specify how many matches to take. A move in Checkers might specify the location of the piece to move and the location to which to move it. Note that a particular value of type `move` might not be legal in a particular game state. For example, for Tic-tac-toe, a move might be an integer from 1 to 9 indicating which square is to be marked. Note that the move `1` is still a move even in the case where the upper left square of the board has an “X” in it – it’s just not a *legal* move!
- A procedure to initialize the game board:
 - `initialState`: Takes in an initializing string, and produces a state. We have provided the stencil code for you, which we’ll explain in more detail in the next section.
- Type conversion procedures: How does one go from internal representations to an external representation as a string (and vice versa, in the case of `move`)?
 - `stringOfPlayer`: Produces the string representation of a player.
 - `stringOfState`: Produces a string representation of a game state: what information do both players need to know in a turn? For tic-tac-toe, this might produce a string that when printed actually shows a 3×3 board with lines dividing up the squares, etc., and a note at the bottom indicating whose turn it is.
 - `stringOfMove`: Produces a string representation of a move.
 - `moveOfString`: Produces an internal representation of a move, given a string and state. This involves parsing a string and failing if the input is not valid or legal.
- Game logic: These are the operations that define your game.
 - `legalMoves`: Produces a list of all the legal moves at a given state.
 - `gameStatus`: Produces the status of the game at a given state.
 - `nextState`: Given a state and a move, produces the state that results from making that move.
- A static game-state evaluator
 - `estimateValue`: provides an estimate of the value, to player 1, of a particular state in the game. This value should be positive for states where player 1 seems to be winning (and especially for those in which player 1 has actually won!), negative for those in which player 1 seems to be losing (and especially for those in which player 1 has actually lost), and have intermediate values for game-states in which the advantage to player 1 is not so clear. A very simple `estimateValue` assigns the value 1.0 to each winning state for player 1, -1.0 to each losing state for player 1, and 0.0 for all other states. Your actual `estimateValue` should be more discriminating than this. Also: the values need not be between -1.0 and 1.0: all that matters is that larger values mean that a situation is better for player 1. Now think about what these values mean for player 2. The name `estimateValue` arises because of the way that this particular procedure gets used by the minimax algorithm.

Game Module Your next task is to write a module that implements this `Game` signature for Connect 4. Here's a road map for how you might proceed:

1. Write a template for a module that implements a testable version of the game, starting like this:

```
module Connect4 = {
  type whichPlayer = P1 | P2;
  type status = Win(whichPlayer) | Draw | Ongoing(whichPlayer);

  type state = ...

  type move = ...
  ...
};
```

2. Fill in the types for `state` and `move`. We have filled in the `whichPlayer` and `status` types for you.
3. Add **stubs** for all of the procedures in the `Game` signature. A stub is just a procedure header plus a body which does nothing useful, or reports an error. For example, here's a stub for the `legalMoves` procedure:

```
let legalMoves: state => list(move) = fun
| _ => failwith("not implemented");
```

4. The `Connect4` module is concrete. You should leave it concrete so that your helpers and types and values are fully visible, making it easier to test them.

However, you should ensure that your `Connect4` module implements the `Game` signature. To do this, use the `open` directive to read in the `Game` signature (in `Game.re`). Finally, try making an abstract module from your `Connect4` module using signature ascription, e.g.

```
module MyGame : Game = Connect4 ;
```

or, equivalently,

```
module MyGame = (Connect4 : Game) ;
```

If this succeeds, your module is consistent with the `Game` signature. If not, the error message should give you a hint as to what is missing or wrong.

5. Gradually fill in all of the stubs you just wrote until they're working, *except* `estimateValue`.

Note that we have provided you with a stencil for the procedure `initialState`, which takes in an initializing `string` (in the case of Connect 4, representing the dimensions of the game board) and produces a `state`, which for Connect 4 would include a board with those dimensions. In particular, for the game Connect 4, our board has two relevant dimensions: `boardHeight` and `boardWidth`, both of which are represented as `int`s. A board of height 5 and width 7, for example, would be input to `initialState` as the string "5 7", where height is the first number in the string, width is the second number in the string, and each dimension is separated by a space.

We've provided some string parsing code for you in `CS17SetupGame.re`, which turns the string "5 7" into a list of integers, and then extracts the relevant height and width values. Thus, our stencil looks like this:

```
let initialState: string => state =
  s => {
    let boardDims = parseBoardDims(s);
    let boardHeight = getBoardHeight(boardDims);
    let boardWidth = getBoardWidth(boardDims);
    /* your initial state, using boardHeight and boardWidth, goes
    here */
  };
```

All you have to worry about is using `boardHeight` and `boardWidth` to construct your board and state!

You may be wondering, why use a `string` to represent the board dimensions? Wouldn't it be easier to use a tuple consisting of two `int`s? That would definitely be easier for the game Connect 4! However, remember that the `Game` module can be used to implement any two-player, sequential, finite-action, deterministic, zero-sum game of perfect information. For example, in the game Nim, which you have seen (or will see) in lab, we don't have a game board with dimensions – instead, we initialize the game with some number of matches, so our initializing string might consist of the single number "21". Using a string, which can then be parsed in the `Game` module implementation, means that our signature can still work for many kinds of games!

At this point, you're almost ready for two human players to play your game.

(See Section 3 for instructions.)

2.3 Player Signature and Modules

The second major part of this project is to implement an AI player that can intelligently play any game that implements the `Game` signature. To do this, you will implement the minimax search algorithm discussed in lecture, which looks ahead several turns and selects the move that appears to be most beneficial in the long run. But first, let's do something simpler: understand the `Player` signature, and create a `HumanPlayer` that conforms to this signature.

Here's the signature for a `Player`:

```
module type Player = {
  module PlayerGame: Game;
  /* given a state, chooses a move */
  let nextMove: PlayerGame.state => PlayerGame.move;
  let playerName: string;
};
```

A player's only task is to figure out its next move, given the state of the game.

As you can see in the first line, a player has a submodule that satisfies the `Game` signature. Why does it make sense for a player to have this? A player needs to know what game it is playing — this is the submodule.

In `HumanPlayer.re`, we provide a `HumanPlayer`.¹

Your first player-related job is to understand the `HumanPlayer` implementation by reading through it and seeing how it interacts with the Game module functions. While `HumanPlayer` is complete, it will still rely on functions *you* write to fully work.

2.3.1 Parsing Human Input

There's one problem you'll encounter here (in `moveOfString`): what do you do when you ask the user to type a move (e.g., a number from 1 to 3) and the user types "ian" rather than "1", "2", or "3"? Well, you want to ask the user to retype the move, but perhaps your `moveOfString` procedure has already tried to convert "ian" to a number between 1 and 3, been unsuccessful, and hit a "failwith" case, or maybe `string_of_int` has failed by "raising an exception." At this point, your program has halted, and there's nothing you can do.

You should address this with something called a "try" expression. Here's an example. Let's start with a simple procedure:

```
let f = fun
  | x => string_of_int(1000 / x);
```

The procedure can be applied to arguments in the usual way:

```
f(25);
f(0);
```

The first invocation is successful:

```
let f : int => string = <fun>;
- : string = "40"
```

The second is not:

```
Exception: Division_by_zero.
```

As you can see, when Reason evaluated $f(0)$, there was a divide-by-zero exception raised (i.e., there was a problem, and something in the Reason interpreter noticed it, and did something called "raising an exception", which ended up halting the program and printing a message).

The structure of a try expression is

```
try (<exp>) {
  | E1 => ...
  | E2 => ...
}
```

where `<exp>` is some expression; if it evaluates without exceptions to some value v , then the value of the try-with expression is just v . On the other hand, if there's an exception, we can "pattern-match" the exception to produce a different value (and to prevent program termination). Here's a modified version of our procedure, and the new results:

¹Not literally, of course! We provide an implementation of `Player` that determines its next move based on user (i.e., human) input.

```
let f = fun
  | x =>
    try (string_of_int(1000 / x)) {
      | _ => "Divide by zero"
    };

f(25);
f(0);
```

```
let f : int => string = <fun>;
- : string = "40"
- : string = "Divide by zero"
```

The code “caught” the divide-by-zero exception and provided a replacement value. While this seems like a good thing, if you look at the replacement value, it’s not going to be very useful to any procedure that calls this one. Good exception handling requires considerable thought.

In the particular situation we’re considering, the right response to bad input is to tell the user what was wrong, and ask for a new input. That’s what the try-expression in `HumanPlayer.re` does, relying on the errors thrown in `moveOfString` to account for invalid moves. Once you implement `moveOfString`, try typing in something like “k” instead of an integer. Rather than halting, your program should continue to prompt you to input an integer.

The `nextMove` procedure of a `HumanPlayer` module only handles `Failure`s, not `Exception`s. This is partly to enforce that we’d like you to fail with your own custom error messages for a user. Also, this way, actual `Exception`s still fail – for instance, pressing CTRL-D (control D) while reading input raises an `Exception`, and we’d like to make sure the program still fails then, so you can safely exit the game even when asked to enter a move.

Note: If you want to participate in the in-class tournament to see how your AI stacks up against other students’ game implementations, make sure the “move” your user inputs is just a single integer.

2.4 The AI Player

Now we come to the interesting part: writing code that will play the game effectively.

Here is the syntax for creating an AI module of type `Player`:

```
module AIPlayer = (Game: Game) => {
  module PlayerGame = ...
  ...
};
```

Again, you’ll likely want to ensure that your `AIPlayer` module implements the `Player` signature; to do this, you’ll want to do something similar to what we did for the `Game` signature and `Game` module.

As noted above, you only have to implement one procedure, `nextMove`. But implementing this procedure is not necessarily straightforward. Here’s a road map for how you might proceed:

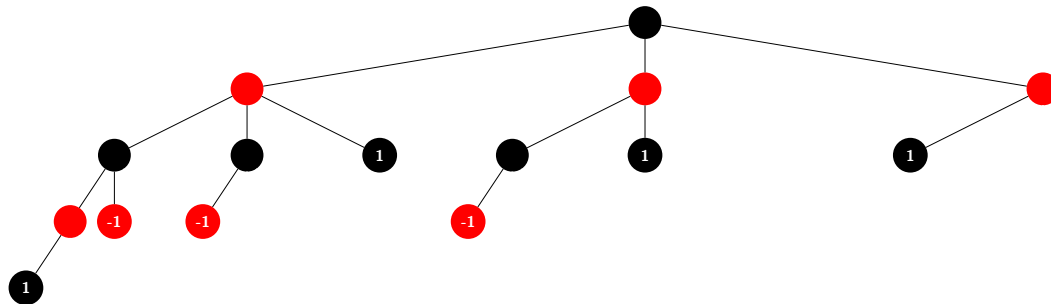
1. In the `AIPlayer` module, replace the stub for `nextMove` with a procedure that simply gets the set

of legal moves and returns any one of them. Verify that you can play a game against your AI player, and that everything works fine (except that the AI player makes naive moves).

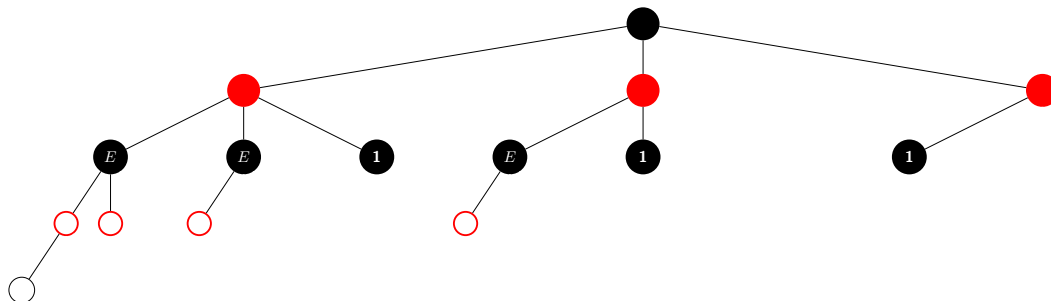
2. Make a first improvement of `estimateValue`, having it return the max value if P1 wins, the min value if P2 wins, or 0 otherwise.
3. Rewrite `estimateValue` to do something sensible. The specifics of this procedure will depend on the game that you choose. This is the hardest part of the assignment for most games, and the one that defines how good your AI is.
4. Write a new version of `nextMove` which first uses `estimateValue` to evaluate the results of each possible move, and then plays the best one. Try out this new version—it should be a little smarter.
5. Implement limited-depth minimax search. You should look ahead something like 3 to 5 levels. (If your game has many possible moves, deep lookahead will be infeasible. If it has only two or three at each state, then quite deep lookahead may be feasible, as long as `estimateValue` is reasonably efficient.) Typically, extra levels of look ahead are worth much more than an improved `estimateValue` procedure, as long as `estimateValue` is at all reasonable.

You can imagine a huge tree, where each node is a possible state of the game, and each edge leaving a node corresponds to a move by one player or the other. A leaf in this “game tree” is a state in which the game is over, either because player 1 or player 2 has won, or because it’s a draw. When the AI player has to make a move, one way to decide on a move is to look at all possible future states of the game (i.e., the subtree rooted at the current state) and see which move leads to the best possible outcome.

But looking at that whole subtree may not be feasible. Instead, if we look only at the current state, its children, and its grandchildren, we have a depth-3 tree sort of sitting “inside” the game tree. We could look at this smaller tree, and try to decide how to move based on it. To do so, we try to pick a move with the property that when the opponent picks their best move, we still end up in the best possible position. (This is the one-level version of minimax.) For this to work, we need to evaluate how much we like a particular “grandchild” state, i.e., we need to *evaluate a leaf* of our tiny tree to see how good a state it is. For that, we use the `estimateValue` procedure in the game.



Above is the game subtree for a NIM game in which there are four pieces left, and it is your turn. The leaves store a value corresponding to the status of the game: 1 if player 1 wins, -1 if player 1 loses.



The bolded nodes constitute the tree that your minimax algorithm will consider, i.e. not necessarily the whole tree. *E* is an estimate for the value of the game given that state.

Note: The AI player can only interact with the game module through the `Game` signature. It's not allowed to access any internal representation of states or moves or anything else. So for an AI player to decide on a move, it must ask the game what are the possible moves, and the resulting next states, and then the moves from *those* states, and so on. It must evaluate the resulting states to see which ones it likes or dislikes, and move accordingly.

2.5 How Everything Fits Together

In our support code, we have provided you with nine files: `Game.re`, `Player.re`, `Connect4.re`, `HumanPlayer.re`, `AIPlayer.re`, `Referee.re`, `ReadLineSyncTest.re`, as well as the setup file and an empty `README` which you should fill out. The first two code files provide the `Game` and `Player` module signatures. They contain all of the types and function type signatures that you will implement in `Connect4.re`, and `AIPlayer.re`. We'll now walk through how the modules fit together in the four main files; remember that we only provide `ReadLineSyncTest.re` for you to ensure that you have correctly installed `readline-sync`, and you do not have to hand it in.

The large-scale structure of the completed project is this: There's a "Referee" module, which incorporates a "Game" module and two "Player" modules. The main procedure in the referee module creates a game (using the `Game` module), and then alternately asks the two players to make moves in that game, keeping track of whether one of them wins. Each "Player" module describes either a human player or an AI player. Depending on which player modules are selected, your "Referee" module should be able to run a game between two humans, or between a human and an AI, or between two AI players.

`Connect4.re` contains the skeleton for a concrete implementation of the `Game` signature, which is located in `Game.re`. All of the details of your game implementation will reside in this concrete module. You will also write test cases in this file, right after the module definition.

In `HumanPlayer.re` and `AIPlayer.re`, we use the same idea to create two concrete implementations of `Player`, `HumanPlayer` and `AIPlayer`, respectively. Notice that in `Player.re`, we specify that `Player`s contain a submodule of type `Game` called `PlayerGame`, allowing use of functions from the `Game` signature to assist in our `nextMove` function. Thus, both `HumanPlayer` and `AIPlayer` must contain a `Game`, which is demonstrated in the following lines:

```
module HumanPlayer = (MyGame: Game) => {  
  module PlayerGame = MyGame;
```

(Note that the first few lines for `AIPlayer` are nearly identical.)

You'll notice that `HumanPlayer` is a functor. A functor takes in a module and uses it to produce another module. In this case, the module the `HumanPlayer` takes in is the `MyGame` module (of type `Game`), representing the game that it is playing.

This means that, to create a `HumanPlayer` for a game of `Connect4`, you'd have to write something like

```
module TestGame = Connect4;  
module HumanConnect4Player = HumanPlayer(Connect4);
```

Notice that you can't use `let` to create a new `Player`; you have to use the "module" keyword to create one.

The `module PlayerGame = MyGame` line inside the module definition defines the `PlayerGame` submodule should be `Connect4.re`.

At this point, to reference any `Game` procedures here, you'll need to use the *qualified name*, i.e. the name must be preceded with `PlayerGame` and a dot. For example, to refer to `legalMoves`, you'll have to write `PlayerGame.legalMoves`. (If you get tired of using qualified names, you can add the command `open PlayerGame` and then use the unqualified names.)

Finally, we have `Referee.re`, which will conduct the actual game play.

3 Let's Play!

As we said earlier, the `Referee` module is in charge of running games. The referee is a functor. It takes in a `Game` and two `Players` that both play the given `Game`.

```
module Referee =  
(  
  MyGame: Game,  
  Player1: Player with module PlayerGame = MyGame,  
  Player2: Player with module PlayerGame = MyGame,  
) =>  
...
```

To let two `HumanPlayers` play one another at `Connect4`, you'd write something like:

```
module Ref = Referee(Connect4, (HumanPlayer(Connect4)), (HumanPlayer(  
  Connect4)));
```

Think about what you would write instead if you want to pit a `HumanPlayer` against an `AIPlayer`, an `AIPlayer` against a `HumanPlayer`, or two `AIPlayers` against each other.

To start a game, invoke the `playGame` procedure in the resulting module, i.e.

```
Ref.playGame();
```

Note that the argument to `playGame` is of type `unit`, i.e. it is a zero-element tuple.

To do this, bring your code into the environment by using `open` directives. Next, construct a module by applying the functors. Finally, invoke the `playGame` procedure.

```
module Ref = Referee(Connect4, (HumanPlayer(Connect4)), (HumanPlayer(  
  Connect4))); /* change as necessary */  
  
Ref.playGame();
```

You can do something similar if you want to get an `AI` player involved. Just remember that you have to bring your `AI` player functor into the environment using a `open` directive.

4 Getting Started

Here's an overview of the major steps that need to be taken in order to complete the project. This is one possible path you could follow, but feel free to complete the project in whichever order works best for you and your partner. For more specific information, consult the rest of the PDF, Ed, or your friendly TA staff!

- You should start by defining the attributes and rules in `Connect4.re`. This includes things like defining your board, valid states, and legal moves. Ensure that all the methods and abstract data types in `Connect4.re` all implement the signatures in `Game.re`. **Note that you should not modify `Game.re`.**
- Next steps may be to fill in `AIPlayer.re`, which will contain the bulk of the artificial intelligence infrastructure for your game (minimax in particular).
- Test your game, not only in the traditional “checkExpect” fashion of CS17, but also ensure that the game behaves as expected in all four Game modes (Human vs. Human, Human vs. AI, AI vs. Human, AI vs. AI). If you can't beat your AI, that's a good sign! Note that you should test `HumanPlayer.re`. One way to approach this is by creating situations in which the next move is obvious, and making sure that your AI picks the correct move.

5 Testing

Since you cannot access the internal implementation details of the Game module in the Player modules, you need not unit test the `nextMove` procedure, because you can neither construct the input board nor an output board to compare to. Instead, you should implicitly test `nextMove` by playing against your AI/watching games between two AIs to ensure that they are choosing reasonable moves (blocking when appropriate, moving to win when possible). If you write any helpers in the player module that do not deal directly with game state, you should test those helpers using `checkExpect`s.

You can and should write test cases for any helpers that do not take in or produce game states in AI player.

Note that we do not provide you with any special `checkExpect`s in our setup file. This is because how you implement your types, such as `state`, is up to you! If we tried to write a `checkExpectState`, it might not work for everybody's representation of a `state`. This means that you should use the regular `checkExpect` for all of your test cases. Where possible, we recommend using `stringOfPlayer`, `stringOfState`, and `stringOfMove` to write your `checkExpects`.

6 Notes

- There are some constructs that Reason will let you use but that we, the CS 17 instructors, will not. Specifically, you are not allowed to use mutation. (If you don't know what this means, don't worry; you won't use it by accident.) As always, if you are unsure about whether a certain Reason construct is allowed, ask the TAs.
- The Reason `List` module has a number of procedures that should come in handy. Documentation can be found online at:
<https://reasonml.github.io/api/List.html>.
Documentation for the Reason Pervasives module (i.e., the procedures that are available in any Reason program) can also be found online at:
<https://reasonml.github.io/api/Pervasives.html>.

Old homeworks, labs, and lecture notes may also contain useful procedures.

- Here are some things you should know about strings:
 - The infix operator `++` concatenates two strings.
 - In a string, `\n` (backslash n) represents a newline character. For example, if the string

```
"hello\nworld"
```

 is printed out,² it will have a line break in between “hello” and “world”.
 - `string_of_int` and `string_of_float` return a string representation of the number passed to them.
 - `int_of_string` and `float_of_string` return a number based on the string passed to them.

7 Handing In

7.1 Design Check

Design checks will be held from November 19th to November 21st. By now, you should have received an email from the HTAs containing information about your design check group, as well as an email with information about how to sign up for design checks. You must sign up for a design check by **12:00 pm EDT on Thursday, November 17th**. We recommend getting in communication with your partner(s) as soon as possible so that you can sign up for a design check time that works for all of you.

There are three components to the design check.

First, you should have at least a partial implementation of the Game signature (i.e. your `Connect4.re` file). You should fill out the type definitions for `move` and `state`, write the `initialState` value.

In addition, you should begin to think about how you want to implement `nextState`, and how you want to design `estimateValue`.

What you have should be well written, organized, and commented. You should be able to explain everything you’ve written to your TA.

Second, you should be able to describe how you intend to implement minimax. Specifically, you should be able to answer the following questions:

- Which procedures in your `Connect4` module will the AI player use? Why?
- What should the AI player do if there are no legal moves available? If you don’t think this should ever happen, tell us why not.
- How does your AI player take into account the fact that the other player wants the AI player to lose? Does your strategy work even if two AI players are playing against each other?

Third, make sure you understand how the support code works and fits together.

²Strings will include these special characters just as you typed them; to print them out, use the `print_string` operation. Unlike all the procedures we have written this semester, this operation does not only return a value; instead, it does something—namely, it prints out a message. We wrote the `HumanPlayer` for you because it uses operations like this one that aren’t purely functional (because it has side-effects other than the value it returns; in this case the procedure returns a unit, `()`, an empty value in Reason).

- How does the HumanPlayer code work? What does it rely on? You will be asked to walk the TA through a brief explanation of the HumanPlayer code to prove that you understand it.
- How does the Referee code work? Like with HumanPlayer, you'll be asked to walk the TA through a brief explanation of the Referee code to prove that you understand it.

This is the minimum that we're expecting, but feel free to do more! Remember, this is your opportunity to have the TAs look closely at your code, and to ask them questions about it.

Before your design check meeting, upload your notes and your `Connect4.re` file to Gradescope. You must upload your files before your design check, even if that slot is before the official Gradescope deadline. Only one partner should hand in the files.

During the design check, you will discuss your solutions to each of the questions above with a TA. Please come prepared! The better prepared you are, the more productive the design check will be. Please also come on time — arriving late to your design check or final grading may result in a deduction.

7.2 Final Handin

The final hand-in is due by 11:00 PM, December 5th, 2022.

For the final hand-in, you are required to hand in eight files: a `README.txt` file, a `Connect4.re` file containing your game implementation, a `HumanPlayer.re` file, containing the Human player implementation, a `AIPlayer.re` file, containing your AI player implementation (including minimax), a `Referee.re` file, containing the Referee module, a `Game.re` file containing the game signature, a `Player.re` file containing the player signature, and a `CS17SetupGame.re` file.

In the `README` file, you should provide:

- instructions for use, describing how a user would interact with your program (how would someone play your game against a friend? against the AI?)
- an overview of how your program functions, including how all of the pieces fit together
- a description of any `possible bugs` or problems with your program
- a list of the people with whom you collaborated
- a description of any extra features you chose to implement

Hand in your files via Gradescope. Only one partner has to submit the project.

7.3 Grading

The design check counts for 20% of your grade. Specifically,

- Your draft implementation of the `Connect4` module: 10 points
- Your plans for implementing the `AIPlayer` module: 6 points
- Your walkthrough of the `HumanPlayer` and `Referee` modules: 2 points.

- Balanced partner contribution: 2 points

Functionality counts for 60% of your score. Specifically, we will look for:

- An initial state with customizable parameters
- Correctly generating legal moves
- Correctly determining the status of the game
- Correctly transitioning to the next state
- Correctly providing support for human input
- Estimating the value of a game state well

Your AI Player will count for 20% of your grade. Specifically, we will look for:

- A correct implementation of minimax
- Making intelligent moves

Partial functionality merits partial credit.

Note that style is being graded deductively. Remember to follow the design recipe; that is, include type signatures, specifications, and test cases for all procedures you write. And add comments to any code which would otherwise be unclear.

After your work has been graded, you and your partner will have the option to attend a virtual final grading meeting. Final grading meetings will take 30 minutes during which, you and your partner(s) and a TA will discuss how your project was graded and any mistakes you made.

As a part of your final grading, you have the opportunity to fix any functionality mistakes on your hand-in and then resubmit the project on Gradescope. This resubmission process can get you up to 50% of credit back on any functionality points that you did not earn. You can only work on the resubmission during the final grading meeting itself and up to 30 minutes after the end of your final grading meeting. To qualify for resubmission points, all partners must attend the final grading. Furthermore, you must submit to the resubmission Gradescope assignment within 30 minutes of the end of your final grading meeting.

A TA will contact you with more information about final gradings after the assignment deadline has passed.

8 Connect 4 Tournament (Optional)

At the end of this project, the TAs will be running an automated tournament pitting your Connect 4 AIs against each other in a mind-bending computerized showdown! Participation in the tournament is completely voluntary, but if you'd like to do so, you must

- Represent your `move` type as an integer between 1 and the number of columns in the board, 1 indicating a move into the leftmost column, and each successive integer indicating a move into the next column to the right

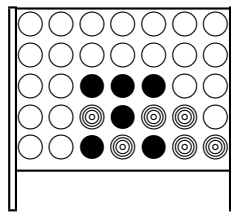
- Begin your Connect4 by defining `initialRows` to be 5 and `initialCols` to be 7, and using those variables in your `initialState` instead of hard-coding them.
- Give your `AILPlayer` a name in the `playerName` value of the module. Even if you don't plan on participating in the tournament, you should still make sure to give your player a name so that it matches the module signature. You can be as creative as you want (provided that your name is still appropriate) — one of our favorites from last year was “CS17 is my villain origin story.”

A Connect 4 Summary

Connect 4 is a game of brutal conflict. If you choose to implement this game, you'll be able to pit the skills you've honed for years against the cold heart of an AI player.

Connect 4 is played on a grid. One player is red and the other black. The players take turns dropping red and black checkers into the grid. The grid can be filled only from the bottom up. The red (black) player's goal is to line up four of red (black) checkers in a row.

Figure 1: A 5×7 Connect 4 game. Black has a guaranteed win.



Parameters While standard Connect 4 is played on a 6×7 board (but we refer to the board in this handout as 5×7 to make your AI move faster because they have to search less possible game states), you should make the size of your board configurable, allowing for play on any board with at least 4 rows and at least 4 columns. If you want an extra challenge, you can also make configurable the number of consecutive checkers required to win.

Strategy The Connect 4 strategy can be summarized fairly simply: always try to extend a line, and if possible, to leave both ends of the line open. If you can manage to make 3-in-a-row with empty slots at both ends, you have a guaranteed win. More generally, if with one move you can complete two different 3s-in-a-row, each with an empty slot at one end (and the empty slots are distinct), you're likewise guaranteed to win. As a special case of this, the trademark Connect 4 strategy is the one where you set up the two empty slots to be vertically adjacent (as in Figure 1); thus in blocking one win, your opponent sets you up for another!

B Other Optimization Method

This extra optimization is not required by any means; if you finish early and want to do more optimization, feel free to use this as a starting point.

Next State One optimization method for your AI player may rely on your `nextState` procedure.

Key idea: You want to be sure that you are looking at every possible game board when building your game tree.

When your AI player is making a move, it should think: if I move here, this will be the next state of the game board; now from this next state, here are all the moves that I can make, and if I move here then this will be the subsequent next state. By looking at every possible game board and every possible combination of moves, your `estimateValue` procedure will be more effective when assigning a move to a specific value as it will know the best combination of moves that can be played.

Please let us know if you find any mistakes, inconsistencies, or confusing language in this or any other CS17 document by filling out the (mostly) anonymous feedback form [here!](#)